

# Variable Selection using Artificial Neural Networks

Maxime Van Cutsem

MAXIME.VANCUTSEM@UNIGE.CH

Editor: XXX

## Abstract

Variable selection in neural networks remains challenging due to their nonlinear and highly parameterized structure, in particular because first-layer weights cannot be interpreted independently of the downstream network. Yet, selecting a relevant subset of input variables can improve interpretability while reducing model size. We introduce an adaptive first-layer regularization framework for multilayer perceptrons relying on a single regularization parameter. Each first-layer neuron is weighted by its downstream sensitivity, yielding a scale-invariant penalty and normalizing the corresponding first-layer gradients. The penalized problem is solved on the representatives of the rescaling symmetry where every sensitivity equals one, where it becomes a lasso on the first layer, by a proximal adaptive gradient method whose zero-thresholding test is the calibrated one. To preserve variables whose relevance emerges only through nonlinear interactions, the regularization level is raised progressively along a warm-started path. The resulting sparse networks discard irrelevant input variables and their associated parameters, thereby reducing both input dimensionality and model complexity. We develop the framework for Gaussian regression, binary classification, and Cox proportional-hazards models, and investigate its variable-selection and sparsification properties through numerical experiments.

**Keywords:** neural networks, variable selection, sparse neural networks, adaptive regularization, pivotal loss; proximal optimization

## 1 Introduction

In machine learning, models ranging from linear methods to artificial neural networks (ANNs) have achieved remarkable success: the former owing largely to their simplicity and interpretability, and the latter to their flexibility and ability to learn complex relationships from data. Selecting which features a model should be allowed to use, however, is a statistical problem before it is a computational one. In the spirit of Occam’s razor—*“Entities must not be multiplied beyond necessity”*—among models with comparable predictive performance, preference should be given to those that achieve it with fewer parameters. Yet modern machine learning has increasingly favoured predictive performance and large-scale parametrization over parsimony. As computational power and storage have become more readily available, it has become feasible to train increasingly large models using many inputs and parameters, often without explicitly questioning whether all of them are necessary. In this regime, overparameterization is computationally affordable, but computational feasibility does not imply statistical necessity. A neural network may therefore achieve strong predictive performance while relying on an unnecessarily large set of inputs and parameters, making it difficult to distinguish the variables that genuinely contribute to the model from those that are merely accommodated by its flexibility. As a result, the input-layer weight

matrix  $W^{(1)}$  may remain unnecessarily dense: irrelevant input variables retain connections to the hidden layer even though these connections are not needed for prediction. Identifying and removing such variables therefore not only improves interpretability but can also substantially reduce the number of parameters required by the network.

For linear models, this variable-selection problem is well posed and extensively studied. The lasso of Tibshirani (1996), for instance, produces a family of supports indexed by a regularization parameter  $\lambda$ , and a substantial literature addresses how this parameter should be chosen. In neural networks, the same question arises in a fundamentally different setting: the parameterization is non-identifiable, the training objective is non-convex, and the fitted model is typically obtained through first-order iterative optimization rather than by solving the optimization problem to global optimality. Sparsity-inducing penalties on the input layer extend the principle of variable selection to this setting (Feng and Simon, 2019; Lemhadri et al., 2021), but the crucial choice of  $\lambda$  is still most commonly delegated to cross-validation.

Cross-validation is unattractive for variable selection for two reasons. First, it is computationally expensive, as it multiplies the cost of an already demanding training procedure by the number of folds and candidate values of  $\lambda$ . More fundamentally, it targets predictive risk rather than variable-selection accuracy: the value of  $\lambda$  that minimizes prediction error need not recover the underlying support and may retain variables carrying little or no signal. If the objective is variable selection, the regularization level should instead be calibrated directly against the event one seeks to control—the inclusion of variables carrying no predictive signal. A natural approach is therefore to calibrate  $\lambda$  under the pure-noise model, in which no variable carries predictive signal, so that, for a prescribed  $\alpha$ , no variable is selected with probability at least  $1 - \alpha$ . Such null-based calibration has been developed for Gaussian linear regression (Donoho and Johnstone, 1994; Belloni et al., 2011) and, more recently, extended to a broader class of loss functions through the pivotal information criterion of Sardy et al. (2026). The pivotal construction makes the null distribution of the calibrating statistic independent of unknown nuisance parameters, allowing  $\lambda$  to be calibrated without estimating quantities such as the noise scale. In this work, we adopt this pivotal framework and the corresponding loss functions considered in Sardy et al. (2026).

Transferring this calibration principle to neural networks introduces an additional difficulty. The gradient with respect to the input-layer weights depends on the downstream network weights, which, because neural-network parameterizations are non-identifiable, can be rescaled without changing the function represented by the network while simultaneously changing the magnitude of the input-layer gradients. Consequently, a regularization threshold based directly on these gradients is itself parameterization-dependent: the same fitted function may cross or remain below the threshold solely as a consequence of rescaling. A meaningful null calibration for neural networks must therefore account for the arbitrary scale induced by the network parameterization.

This paper addresses this difficulty by exploiting the structure of the input-layer gradient under the null. We show that, when the input layer is empty, this gradient is exactly a rank-one matrix: the outer product of a network-sensitivity vector and a data-dependent vector. The data-dependent factor carries the evidence against the null and is calibrated once, before training, by Monte Carlo, whereas the sensitivity factor captures the current scale of the network and is recomputed at every iteration. This decomposition yields an

adaptive regularization level that follows the scale of the network throughout training, making variable selection invariant to equivalent rescalings of its parameterization while preserving the original null calibration. Following the pivotal framework described above, we focus on three tasks: regression, binary classification, and Cox survival analysis.

The resulting method is simple. Once every neuron is rescaled so that its sensitivity equals one, the penalized problem becomes a lasso on the first layer under a normalization constraint. Its optimality conditions are those of a lasso on the first layer, and of a Lagrangian with closed-form multipliers on the other parameters (Section 3). The calibration of  $\lambda$  reads the lasso condition at the null model (Section 4), and the training algorithm takes one step on each: a proximal lasso step on the first layer, and a gradient step on the Lagrangian for the other parameters (Section 5).

## Contributions.

- A zero-thresholding condition for a penalized multilayer perceptron (Theorem 6) and an exact rank-one factorization of the null gradient (proof of Theorem 6) separating the statistical evidence from the network amplification.
- An adaptive penalty,  $\lambda \sum_k s_k(\boldsymbol{\theta}) \|W_{k,\cdot}^{(1)}\|_1$ , in which only  $\lambda$  is calibrated. It is exactly invariant under the rescaling group of the network (Proposition 1), recovers the fixed-architecture condition at the null (Theorem ??) uniformly over the unpenalized parameters, and is the largest admissible such rule (Proposition ??).
- The consequence that the calibrated threshold does not depend on the architecture (Corollary ??): one calibration serves every depth and width, and collapses to the linear-model threshold.
- Pivotal losses and closed-form null gradients for Gaussian regression, binary classification and the Cox model (Table 1), reducing the calibration to a single matrix product per Monte Carlo draw.
- A simple optimization scheme (Section 5): once every sensitivity is set to one, the problem is a lasso on the first layer and an explicit Lagrangian, with known multipliers, on the other parameters. Each iteration costs about one epoch of a standard network, and at the null it performs exactly the calibrated zero-thresholding test (Proposition 8).

## 2 Related Work

**Sparse-input networks.** Penalizing the input layer of a network to obtain feature selection is an established device. Feng and Simon (2019) fit sparse-input networks with a group penalty on the input weights, Louizos et al. (2018) relax an  $\ell_0$  count with stochastic gates, as do Yamada et al. (2020), and Lemhadri et al. (2021) tie the input layer to a residual linear path so that the selected support is nested along the path. In all of these the regularization level is a tuning parameter, chosen by cross-validation or by a target sparsity; the present paper leaves the architecture and the penalty alone and supplies the level.

**Selection with error control.** A second line of work attaches a frequentist guarantee directly to the selected set. Knockoffs (Barber and Candès, 2015) and their model-X extension (Candès et al., 2018) control the false discovery rate by comparing each original variable with an artificial counterpart designed to mimic the dependence structure of the covariates while carrying no additional information about the response. DeepPINK (Lu et al., 2018) extends this principle to neural networks through a dedicated architecture that jointly processes original variables and their knockoffs. These approaches provide a stronger inferential guarantee than the one pursued here, but require the construction of valid knockoff variables, which in the model-X setting relies on knowledge or accurate estimation of the covariate distribution. Our approach instead operates directly on the original covariates and can be integrated into a standard MLP without generating artificial variables or modifying the architecture to accommodate them. The resulting guarantee is different in nature: our calibration controls the zero-thresholding behavior at the null model rather than the false discovery rate of the returned support.

**Tuning without cross-validation.** Choosing  $\lambda$  from the null rather than from held-out risk goes back to the universal threshold of Donoho and Johnstone (1994) and, for the lasso, to the theoretical choice of Bickel et al. (2009). The quantile universal threshold (Giacobino et al., 2017) replaces the asymptotic rate by a finite-sample quantile of the zero-thresholding statistic, and the pivotal information criterion (Sardy et al., 2026) makes that quantile computable by removing the nuisance parameters from the statistic. This paper is the extension of that programme to a nonlinear, non-identified model.

**Rescaling invariance.** That a positively homogeneous network can move mass between layers at no cost, so that a norm read off one layer means nothing on its own, is the phenomenon behind path-normalized measures of capacity (Neyshabur et al., 2015b) and behind Path-SGD (Neyshabur et al., 2015a), which replaces gradient descent by a descent invariant to these rescalings. We take a different route, dictated by what the calibration needs: we keep the  $\ell_1$  penalty on  $W^{(1)}$  and rescale the *level*, by exactly the factor that appears in the null gradient. The optimization then moves, at every step, to the rescaled network where this factor equals one; moves of this kind, which leave the function unchanged, are also used by equi-normalization (Stock et al., 2019) and symmetry teleportation (Zhao et al., 2022). The closest algorithm is PathProx (Yang et al., 2022), which normalizes the input weights of every ReLU neuron and takes proximal steps on the output weights; here the normalizing factor depends on the data and on the deeper layers, which adds one term to the gradient (Section 5).

### 3 Sparse Learning with Neural Networks

#### 3.1 Setting and Notation

Let  $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$  be independent observations, with  $\mathbf{x}_i \in \mathbb{R}^p$ , and let  $X \in \mathbb{R}^{n \times p}$  denote the design matrix. Throughout, without loss of generality, we normalize

$$\frac{1}{n} \sum_{i=1}^n x_{ij}^2 = 1 \quad (j = 1, \dots, p).$$

Consider a fully connected artificial neural network (ANN), also known as a multilayer perceptron (MLP), with  $L$  layers, defining a class of functions

$$f_{\boldsymbol{\theta}}(\mathbf{x}) = S_L \circ S_{L-1} \circ \cdots \circ S_1(\mathbf{x}).$$

Throughout this work, we restrict attention to MLPs with a scalar output, so that  $f_{\boldsymbol{\theta}} : \mathbb{R}^p \rightarrow \mathbb{R}$ . The parameter vector  $\boldsymbol{\theta}$  collects all model parameters. For observation  $i$ , we write  $\eta_i = f_{\boldsymbol{\theta}}(\mathbf{x}_i)$ , and  $\boldsymbol{\eta} = f_{\boldsymbol{\theta}}(X) \in \mathbb{R}^n$  for the vector of outputs over the  $n$  observations. Write  $p_0 = p$  for the input dimension and  $p_l$  for the number of neurons in layer  $l$ , so that  $W^{(l)} \in \mathbb{R}^{p_l \times p_{l-1}}$  and  $\mathbf{b}^{(l)} \in \mathbb{R}^{p_l}$  denote the weight matrix and bias of layer  $l$ . For  $l \in \{1, \dots, L-1\}$ , layer  $l$  applies an affine map followed by a componentwise nonlinearity,

$$S_l(\mathbf{u}) = \sigma_l(\mathbf{b}^{(l)} + W^{(l)}\mathbf{u}),$$

mapping the  $p_{l-1}$ -vector  $\mathbf{u}$  into a  $p_l$ -vector. The output layer is purely affine,

$$S_L(\mathbf{u}) = \mathbf{b}^{(L)} + W^{(L)}\mathbf{u}, \quad p_L = 1,$$

with no activation applied: any transformation of the output, such as a sigmoid for binary classification, is absorbed into the loss so that regression ( $y \in \mathbb{R}$ ), binary classification ( $y \in \{0, 1\}$ ) and survival data are handled by the same architecture.

With an MLP, the relevant features are the columns of the first-layer weight matrix  $W^{(1)}$  that are non-zero, as these are the only ones directly connected to the input  $X$ . For this identification to be meaningful, a small entry of  $W^{(1)}$  must not be reproducible by inflating weights in deeper layers: otherwise an input could influence the output while its first-layer column is arbitrarily small, severing the link between  $W^{(1)}$  and feature relevance and nullifying any penalty applied to  $W^{(1)}$ .

The quantity that governs this amplification is the sensitivity of the output to the first layer. Write  $\mathbf{h}_i^{(1)} = \mathbf{b}^{(1)} + W^{(1)}\mathbf{x}_i$  for the first-layer preactivation at observation  $i$  and define the *sensitivity vectors*

$$\mathbf{a}_i(\boldsymbol{\theta}) = \frac{\partial \eta_i}{\partial \mathbf{h}_i^{(1)}} = D_i^{(1)} W^{(2)\top} \cdots D_i^{(L-1)} W^{(L)\top} \in \mathbb{R}^{p_1}, \quad i = 1, \dots, n, \quad (1)$$

where  $D_i^{(\ell)} = \text{diag}(\sigma'_\ell(\mathbf{h}_i^{(\ell)}))$ . They measure how strongly the network output responds to each first-layer unit, and they are the exact factor by which a gradient signal is amplified between the penalized weights and the output. For each first-layer neuron  $k$ , its *neuron sensitivity scale* is defined by

$$s_k(\boldsymbol{\theta}) = \|\mathbf{a}_{\cdot k}(\boldsymbol{\theta})\|_\infty = \max_{1 \leq i \leq n} |a_{ik}(\boldsymbol{\theta})|. \quad (2)$$

Section 3.2 shows that measuring the penalty on this sensitivity scale removes the arbitrary neuron-wise scaling of the network.

Throughout this work, we use the leaky ReLU activation

$$\sigma_\ell(t) = \max\{t, mt\}, \quad \ell = 1, \dots, L-1, \quad m \in (0, 1),$$

in all hidden layers. This choice is motivated by two properties that are particularly useful for the proposed regularization.

First, the derivative of each hidden-layer activation remains bounded away from zero wherever it is defined. More precisely,

$$|\sigma'_\ell(t)| \geq m > 0.$$

This prevents the activation functions themselves from completely blocking the propagation of information through the network and, in particular, the backpropagated signal used to construct the sensitivity vectors  $\mathbf{a}_i$ . This property is also relevant to the structural interpretation of variable selection through  $W^{(1)}$ . If an intermediate activation contains a flat region, an input–output path may be blocked by the activation even though the corresponding entries of  $W^{(1)}$  are nonzero. The network may then become functionally independent of some input variables without the associated first-layer weights being zero.

Second, the leaky ReLU is positively homogeneous:

$$\sigma_\ell(ct) = c\sigma_\ell(t), \quad c > 0.$$

Although this property holds for every hidden layer, it is specifically required at the first one for the scale-invariance of the adaptive penalty developed in Section 3.2.

### 3.2 The Scale Ambiguity and the Adaptive Regularization

A multilayer network does not uniquely identify its own parameters, and this ambiguity affects precisely the quantity that a penalty on  $W^{(1)}$  is meant to measure. By positive homogeneity of the first-layer activation, for any first-layer neuron  $k$  and any  $c > 0$ , the reparameterization

$$W_{k,\cdot}^{(1)} \mapsto cW_{k,\cdot}^{(1)}, \quad b_k^{(1)} \mapsto cb_k^{(1)}, \quad W_{\cdot,k}^{(2)} \mapsto \frac{1}{c}W_{\cdot,k}^{(2)} \quad (3)$$

leaves the fitted function  $f_\theta$ , and therefore the loss, unchanged, while multiplying  $\|W_{k,\cdot}^{(1)}\|_1$  by  $c$ . Read in the direction  $c \rightarrow 0$ , this is the failure mode announced in Section 3.1: the network drives its input weights towards zero, compensates by inflating  $W^{(2)}$ , predicts exactly what it predicted before, and pays a penalty as small as it likes.

The sensitivity scale (2) transforms in the opposite direction. Indeed, under (3),

$$\mathbf{a}_{\cdot,k}(\theta) \mapsto \frac{1}{c}\mathbf{a}_{\cdot,k}(\theta), \quad s_k(\theta) \mapsto \frac{1}{c}s_k(\theta).$$

The sensitivity scale therefore provides exactly the compensating factor needed to remove this arbitrary parameter scaling.

**Proposition 1** *The adaptive first-layer penalty*

$$\mathcal{P}(\theta) = \sum_{k=1}^{p_1} s_k(\theta) \|W_{k,\cdot}^{(1)}\|_1$$

*is invariant under any positive neuron-wise reparameterization of the form (3). Consequently, any penalized criterion does not depend on the arbitrary scale chosen between a first-layer neuron and the following layer.*

The role of the sensitivity scale extends beyond scale invariance. By matching the sensitivity factor appearing in the first-layer gradient, it removes neuron-specific downstream amplification from the regularization threshold, thereby enabling the calibration of a single regularization parameter  $\lambda$ , as developed in Section 4.

### 3.3 The Learning Problem

Given data  $(X, \mathbf{y})$ , we consider the training problem

$$\min_{\boldsymbol{\theta} \in \mathbb{R}^\gamma} J_\lambda(\boldsymbol{\theta}), \quad J_\lambda(\boldsymbol{\theta}) = \mathcal{L}_n(f_{\boldsymbol{\theta}}(X); \mathbf{y}) + \lambda \sum_{k=1}^{p_1} s_k(\boldsymbol{\theta}) \|W_{k,\cdot}^{(1)}\|_1, \quad (\text{P})$$

where  $\mathcal{L}_n(\boldsymbol{\eta}; \mathbf{y})$  is a data-dependent goodness-of-fit measure of the vector of outputs, and  $\lambda \geq 0$  is the regularization parameter controlling the strength of the adaptive first-layer penalty, set to the level  $\lambda_\alpha^{\text{DB}}$  developed in Section 4. The full expression  $J_\lambda$  is referred to as the cost function. Throughout this work, we consider three statistical learning settings: regression, binary classification, and survival analysis under the Cox proportional hazards model. In each case, the goodness-of-fit term  $\mathcal{L}_n$  is chosen with a specific normalization that will play a central role in the calibration of  $\lambda$ . We consider the same three losses as Sardy et al. (2026).

For regression, with  $\mathbf{y} \in \mathbb{R}^n$ , we use the square-root loss

$$\mathcal{L}_n^{\text{reg}}(\boldsymbol{\eta}; \mathbf{y}) = \frac{1}{\sqrt{n}} \|\mathbf{y} - \boldsymbol{\eta}\|_2.$$

For binary classification, with  $y_i \in \{0, 1\}$  and  $\mu_i = 1/(1 + e^{-\eta_i})$  denoting the logit produced by the network, we use the transformed logistic loss

$$\mathcal{L}_n^{\text{log}}(\boldsymbol{\eta}; \mathbf{y}) = \frac{2}{n} \sum_{i=1}^n \left[ y_i \sqrt{\frac{1 - \mu_i}{\mu_i}} + (1 - y_i) \sqrt{\frac{\mu_i}{1 - \mu_i}} \right].$$

For survival analysis, let  $(y_i, \delta_i)$  denote the observed time and event indicator for subject  $i$ . We use the square-root negative Cox partial log-likelihood

$$\mathcal{L}_n^{\text{Cox}}(\boldsymbol{\eta}; \mathbf{T}, \boldsymbol{\delta}) = \sqrt{\sum_{i=1}^n \delta_i \left[ \eta_i - \log \left( \sum_{j: y_j \geq y_i} e^{\eta_j} \right) \right]}.$$

These particular forms and normalizations are not merely conventional choices. They are selected so that the score of the loss at the null model is a pivotal quantity, free of any nuisance parameters. This property is what ultimately allows our choice of regularization level  $\lambda_\alpha^{\text{DB}}$  developed in Section 4.

### 3.4 The Normalized Problem

Since the rescaling (3) divides  $s_k$  by  $c$ , choosing  $c = s_k(\boldsymbol{\theta})$  brings every sensitivity to one.

**Definition 2 (Normalized network)** Let  $\boldsymbol{\theta}$  satisfy  $s_k(\boldsymbol{\theta}) > 0$  for  $k = 1, \dots, p_1$ . The normalized network  $\mathcal{N}(\boldsymbol{\theta})$  is obtained from  $\boldsymbol{\theta}$  by applying the following transformation to every first-layer neuron  $k$ :

$$W_{k,\cdot}^{(1)} \mapsto s_k(\boldsymbol{\theta}) W_{k,\cdot}^{(1)}, \quad b_k^{(1)} \mapsto s_k(\boldsymbol{\theta}) b_k^{(1)}, \quad W_{\cdot,k}^{(2)} \mapsto W_{\cdot,k}^{(2)} / s_k(\boldsymbol{\theta}).$$

We write  $\Theta_1 = \{\boldsymbol{\theta} : s_k(\boldsymbol{\theta}) = 1, k = 1, \dots, p_1\}$  for the set of normalized networks.

**Proposition 3 (Normalization)** Let  $\boldsymbol{\theta}$  satisfy  $s_k(\boldsymbol{\theta}) > 0$  for  $k = 1, \dots, p_1$ . Then

- (i)  $f_{\mathcal{N}(\boldsymbol{\theta})} = f_{\boldsymbol{\theta}}$  and  $\mathcal{P}(\mathcal{N}(\boldsymbol{\theta})) = \mathcal{P}(\boldsymbol{\theta})$ : the normalized network computes the same function, with the same penalty;
- (ii)  $\mathcal{N}(\boldsymbol{\theta}) \in \Theta_1$ , and  $\mathcal{N}(\boldsymbol{\theta}) = \boldsymbol{\theta}$  if and only if  $\boldsymbol{\theta} \in \Theta_1$ ;
- (iii)  $\mathcal{N}(\boldsymbol{\theta})$  and  $\boldsymbol{\theta}$  have the same zero entries in  $W^{(1)}$ .

**Proof** (i) follows from the positive homogeneity of the first-layer activation and Proposition 1. (ii) The rescaling of neuron  $k$  divides  $s_k$  by  $c = s_k(\boldsymbol{\theta})$  and leaves the other sensitivities unchanged; it is the identity when  $s_k(\boldsymbol{\theta}) = 1$ . (iii) Rows of  $W^{(1)}$  are multiplied by positive factors. ■

Hence, nothing is lost by restricting (P) to normalized networks. On  $\Theta_1$ , the adaptive penalty becomes a plain lasso on the first layer:

$$\min_{\boldsymbol{\theta} \in \Theta_1} \mathcal{L}_n(f_{\boldsymbol{\theta}}(X); \mathbf{y}) + \lambda \|W^{(1)}\|_1. \quad (\text{C})$$

Problems (P) and (C) are equivalent: they have the same optimal value,  $\mathcal{N}(\hat{\boldsymbol{\theta}})$  solves (C) whenever  $\hat{\boldsymbol{\theta}}$  solves (P), and every solution of (C) solves (P). Moreover, by Proposition 3 the selected set  $\hat{S} = \{j : \hat{W}_{\cdot,j}^{(1)} \neq \mathbf{0}\}$  is the same for both. From now on, we work with (C).

### 3.5 Optimality Conditions

Problem (C) has simple first-order conditions, and the rest of the paper rests on them: at the null model, they give the calibration of  $\lambda$  (Section 4); away from it, the algorithm is built to satisfy them (Section 5).

Split the parameters as  $\boldsymbol{\theta} = (W^{(1)}, \varphi)$ , where  $\varphi$  collects every bias and  $W^{(2)}, \dots, W^{(L)}$ . Write  $\ell(\boldsymbol{\theta}) = \mathcal{L}_n(f_{\boldsymbol{\theta}}(X); \mathbf{y})$  for the loss and  $\mathbf{r} = \nabla_{\boldsymbol{\eta}} \mathcal{L}_n(\boldsymbol{\eta}; \mathbf{y})$ , at  $\boldsymbol{\eta} = f_{\boldsymbol{\theta}}(X)$ , for its gradient with respect to the outputs. Since  $\eta_i$  depends on  $W^{(1)}$  only through  $\mathbf{h}_i^{(1)}$ , the chain rule gives the gradient with respect to the  $k$ th row of the first layer,

$$\mathbf{g}_k = \nabla_{W_{k,\cdot}^{(1)}} \ell = X^\top (\mathbf{a}_k \odot \mathbf{r}), \quad k = 1, \dots, p_1. \quad (4)$$

It combines the association between the inputs and the residuals, through  $\mathbf{r}$ , with the downstream network, through  $\mathbf{a}_k$ , whose magnitude is fixed on  $\Theta_1$ :  $\|\mathbf{a}_k\|_\infty = s_k = 1$ .

**Proposition 4 (Optimality conditions of (C))** *Let  $\boldsymbol{\theta} \in \Theta_1$  be a local minimizer of (C) where no hidden pre-activation vanishes, and let  $\mu_k = \lambda \|W_{k,\cdot}^{(1)}\|_1$ . Then*

$$\mathbf{0} \in \mathbf{g}_k + \lambda \partial \|W_{k,\cdot}^{(1)}\|_1 \quad (k = 1, \dots, p_1), \quad \nabla_{\varphi} \ell + \sum_{k=1}^{p_1} \mu_k \nabla_{\varphi} s_k = \mathbf{0}. \quad (5)$$

The proof is given in Appendix A. The two conditions are of a familiar kind.

**The first condition is a lasso.** Near  $\boldsymbol{\theta}$ ,  $s_k$  does not depend on  $W^{(1)}$ , so that the constraint plays no role in the first layer, and the first condition is that of the lasso with loss  $\ell(\cdot, \varphi)$ :  $g_{kj} = -\lambda \text{sign } W_{kj}^{(1)}$  where  $W_{kj}^{(1)} \neq 0$ , and

$$|g_{kj}| \leq \lambda \quad \text{where } W_{kj}^{(1)} = 0. \quad (6)$$

This *zero test* decides which variables are selected: variable  $j$  is discarded when  $|g_{kj}| \leq \lambda$  for every neuron  $k$ .

**The second condition is a Lagrangian with explicit multipliers.** It is the stationarity in  $\varphi$  of the Lagrangian

$$\tilde{\mathcal{L}}_{\lambda}(\boldsymbol{\theta}, \nu) = \ell(\boldsymbol{\theta}) + \lambda \|W^{(1)}\|_1 + \sum_{k=1}^{p_1} \nu_k (s_k(\boldsymbol{\theta}) - 1) \quad (7)$$

at  $\nu_k = \mu_k$ . The multipliers need not be solved for: by the rescaling invariance of  $\ell$ , any local minimizer has  $\nu_k = \lambda \|W_{k,\cdot}^{(1)}\|_1$  (Appendix A). Equivalently, the gradient in  $\varphi$  of the cost function  $J_{\lambda}$  of (P) vanishes. The multiplier term cannot be dropped: with  $\nu = \mathbf{0}$ , the only stationary points have  $W^{(1)} = \mathbf{0}$ , so that no variable could ever be selected.

**Remark 5 (A lasso and a closed-form Lagrangian)** *Proposition 4 reduces (C) to two elementary problems: a lasso in  $W^{(1)}$ , handled by soft-thresholding, and the stationarity in  $\varphi$  of a Lagrangian whose multipliers are known in closed form. Nothing has to be solved for or projected: the constraint is restored at no cost by the normalization of Definition 2, and the only non-standard term,  $\sum_k \mu_k \nabla_{\varphi} s_k$ , costs one extra backward pass on  $p_1$  rows (Section 5.3). The calibration of  $\lambda$  reads the zero test (6) at the null model; the algorithm takes one step on each problem.*

## 4 Choice of $\lambda$

### 4.1 Calibration under the null

The regularization parameter  $\lambda$  in (C) governs the sparsity of the learned model: if  $\lambda$  is too large, the penalty dominates the loss and drives  $W^{(1)} = \mathbf{0}$ , thereby discarding all input features; if  $\lambda$  is too small, irrelevant features may enter the model. A standard approach is to select  $\lambda$  by cross-validation. This is unattractive in the present setting for two reasons. First, repeated fitting is computationally expensive for neural networks. Second, cross-validation targets predictive performance rather than support recovery and may therefore favour insufficient regularization when the objective is variable selection.

We instead build on the null-calibration principle of Sardy et al. (2026). The central idea is to calibrate the regularization level under the pure-noise model  $H_0 : W^{(1)} = \mathbf{0}$  so that, with prescribed probability  $1 - \alpha$ , the solution without any selected variable,  $W^{(1)} = \mathbf{0}$ , is a local minimizer of (C). The first step is to find, for given data, the levels  $\lambda$  that make it one, and Proposition 4 tells where to look: the zero test (6) must hold for every weight.

In a multilayer model, the gradient (4) entering this test depends not only on the statistical association between  $X$  and  $\mathbf{y}$ , through  $\mathbf{r}$ , but also on the downstream network, through  $\mathbf{a}_k$ . The former is determined by the data and can be calibrated under  $H_0$ ; the latter depends on the parametrization of the network. We now show that at the null model the latter disappears altogether, so that a single level  $\lambda$  can be calibrated from the data alone.

Consider the null parameter  $\boldsymbol{\theta}^0 = (W^{(1)} = \mathbf{0}, \hat{\boldsymbol{\tau}}) \in \Theta_1$ , where  $\hat{\boldsymbol{\tau}}$  minimizes the loss subject to  $W^{(1)} = \mathbf{0}$ . Under this constraint the network output is constant across observations, so that  $\hat{\boldsymbol{\tau}}$  yields the optimal constant fit under  $\mathcal{L}_n$ . Let

$$\hat{\mathbf{r}}_0(\mathbf{y}) = \nabla_{\boldsymbol{\eta}} \mathcal{L}_n(\boldsymbol{\eta}; \mathbf{y})|_{\boldsymbol{\eta}=f_{\boldsymbol{\theta}^0}(X)} \in \mathbb{R}^n$$

be the gradient of the loss with respect to the outputs at this fit. At  $W^{(1)} = \mathbf{0}$ , every observation reaches the deeper layers with the same input  $\mathbf{b}^{(1)}$ , so that  $\mathbf{a}_k(\boldsymbol{\theta}^0) = a_{0k} \mathbf{1}$  with  $|a_{0k}| = s_k(\boldsymbol{\theta}^0) = 1$ . By (4), the null gradient is the rank-one matrix

$$\mathbf{g}_{0,k} = a_{0k} X^\top \hat{\mathbf{r}}_0(\mathbf{y}), \quad a_{0k} = \pm 1, \quad k = 1, \dots, p_1 : \quad (8)$$

the network enters only through a sign, and the evidence against the null only through  $X^\top \hat{\mathbf{r}}_0(\mathbf{y})$ .

We can now read the optimality conditions (5) at  $\boldsymbol{\theta}^0$ . The second one holds for every  $\lambda$ :  $\mu_k = 0$  since  $W^{(1)} = \mathbf{0}$ , and  $\nabla_{\varphi} \ell(\boldsymbol{\theta}^0) = \mathbf{0}$  since  $\hat{\boldsymbol{\tau}}$  minimizes  $\ell(\mathbf{0}, \cdot)$ . By (8), the first one is the zero test  $|X_j^\top \hat{\mathbf{r}}_0(\mathbf{y})| \leq \lambda$  for every  $j$ , the same for all neurons. Hence  $\boldsymbol{\theta}^0$  satisfies (5) if and only if

$$\lambda \geq \lambda_0(X, \mathbf{y}) = \left\| X^\top \hat{\mathbf{r}}_0(\mathbf{y}) \right\|_{\infty}. \quad (9)$$

The following theorem shows that this first-order condition is also sufficient for local minimality, as soon as it holds strictly.

**Theorem 6 (Zero-thresholding function)** *Assume that no hidden pre-activation vanishes at  $\boldsymbol{\theta}^0$  and that  $\mathcal{L}_n(\cdot; \mathbf{y})$  is twice continuously differentiable near  $f_{\boldsymbol{\theta}^0}(X)$ , and let  $\lambda_0(X, \mathbf{y})$  be given by (9).*

- (i) *If  $\lambda > \lambda_0(X, \mathbf{y})$ , then  $\boldsymbol{\theta}^0$  is a local minimizer of (C): the level  $\lambda$  forces  $W^{(1)} = \mathbf{0}$ .*
- (ii) *If  $\lambda < \lambda_0(X, \mathbf{y})$ , then  $\boldsymbol{\theta}^0$  is not a local minimizer of (C).*

Part (ii) is Proposition 4: the zero test fails for the variable attaining  $\lambda_0$ . Part (i), proved in Appendix A, is the converse: when every zero test holds strictly, no small move of  $\boldsymbol{\theta}$ , even a joint one in  $\varphi$ , decreases the objective. Moving a weight  $W_{kj}^{(1)}$  away from zero changes the loss at rate  $|g_{0,kj}| \leq \lambda_0$  and the penalty at rate  $\lambda$ , hence the threshold. At

$\lambda = \lambda_0$  the two rates balance and the theorem does not conclude; this case has probability zero under the null distributions considered below.

The zero-thresholding function  $\lambda_0(X, \mathbf{y})$  gives the critical regularization level above which  $W^{(1)} = \mathbf{0}$  is forced for the observed data  $(X, \mathbf{y})$ . To turn this data-dependent threshold into a calibrated regularization parameter, we evaluate its distribution in the setting where the null solution is appropriate, namely under  $H_0$ . This null distribution provides the reference against which the regularization level is calibrated.

**Definition 7 (Detection boundary)** *Let  $\mathbf{Y}_0$  be a response vector generated under the pure-noise model. Define the random variable  $\Lambda = \lambda_0(X, \mathbf{Y}_0)$ , and let  $F_\Lambda$  denote its cumulative distribution function. Then, for a prescribed  $\alpha \in (0, 1)$ , the  $\alpha$ -level detection boundary is*

$$\lambda_\alpha^{\text{DB}} = F_\Lambda^{-1}(1 - \alpha).$$

The resulting  $\lambda_\alpha^{\text{DB}}$  provides a fixed reference threshold calibrated under the null model. By construction,  $\mathbb{P}_{H_0} \{ \lambda_0(X, \mathbf{Y}_0) \leq \lambda_\alpha^{\text{DB}} \mid X \} = 1 - \alpha$ . Combined with Theorem 6, this implies that, under  $H_0$ , the null parameter is a local minimizer of (C) at  $\lambda = \lambda_\alpha^{\text{DB}}$  with probability  $1 - \alpha$ , the event  $\lambda_0 = \lambda_\alpha^{\text{DB}}$  having probability zero. This threshold is determined prior to network optimization and remains unchanged throughout training. Importantly, the downstream network parametrization does not enter its calibration: on  $\Theta_1$ , its contribution to the null gradient reduces to a sign (8). The calibration of  $\lambda$  therefore depends only on the distribution of the zero-thresholding function under  $H_0$ .

In general, such a null distribution may depend on unknown nuisance parameters, preventing the detection boundary from being calibrated without their estimation. The losses introduced in Section 3.3 are chosen precisely to avoid this difficulty. For each of the three learning tasks considered here, the corresponding zero-thresholding statistic  $\Lambda = \lambda_0(X, \mathbf{Y}_0)$  is pivotal under  $H_0$ , meaning that its null distribution is free of unknown nuisance parameters of  $\mathbf{Y}_0$ . Consequently, the detection boundary can be approximated by Monte Carlo simulation under any convenient choice of these nuisance parameters: repeated null samples  $\mathbf{Y}_0^{(1)}, \dots, \mathbf{Y}_0^{(M)}$  are generated, the corresponding zero-thresholding statistics are computed, and  $\lambda_\alpha^{\text{DB}}$  is estimated by their empirical  $(1 - \alpha)$ -quantile.

Table 1 summarizes the resulting zero-thresholding statistic for regression, binary classification, and survival analysis. Further details on the pivotal construction are provided in Sardy et al. (2026), with the survival analysis setting developed specifically in van Cutsem and Sardy (2026).

## 4.2 Interpretation away from the null model

The calibration above reads the zero test (6) at the null model  $W^{(1)} = \mathbf{0}$ , where the sensitivity of each first-layer neuron is constant across observations. Away from the null model, the same test decides which variables are selected. The vector  $\mathbf{a}_k$  now varies across observations, but on  $\Theta_1$  its largest entry is still one in absolute value, and by (4) the condition reads

$$\left| X_j^\top (\mathbf{a}_k(\boldsymbol{\theta}) \odot \mathbf{r}(\boldsymbol{\theta})) \right| \leq \lambda, \quad \|\mathbf{a}_k(\boldsymbol{\theta})\|_\infty = 1.$$

The score is the calibrated one,  $X_j^\top \mathbf{r}$ , reweighted observation by observation by the sensitivity profile of neuron  $k$ , whose overall magnitude is fixed. The level  $\lambda$  therefore acts on a

| Learning task         | Zero-thresholding statistic $\lambda_0(X, \mathbf{y})$                                                                                                          |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Regression            | $\frac{\ X^\top(\mathbf{y} - \bar{\mathbf{y}}\mathbf{1})\ _\infty / \sqrt{n}}{\ \mathbf{y} - \bar{\mathbf{y}}\mathbf{1}\ _2}$                                   |
| Binary classification | $\frac{\ X^\top(\mathbf{y} - \bar{\mathbf{y}}\mathbf{1})\ _\infty / n}{\sqrt{\bar{y}(1-\bar{y})}}$                                                              |
| Survival analysis     | $\frac{\ n^{-1} \sum_{i=1}^n \delta_i(\mathbf{x}_i - \bar{\mathbf{x}}_{\mathcal{R}_i})\ _\infty}{2\{n^{-1} \sum_{i=1}^n \delta_i \log  \mathcal{R}_i \}^{1/2}}$ |

Table 1: Pivotal zero-thresholding statistics for the three learning tasks considered in this work. For the survival analysis task,  $\mathcal{R}_i = \{j : y_j \geq y_i\}$  and  $\bar{\mathbf{x}}_{\mathcal{R}_i} = |\mathcal{R}_i|^{-1} \sum_{j \in \mathcal{R}_i} \mathbf{x}_j$ .

normalized first-layer score and can be shared across all first-layer neurons, away from the null as at the null.

## 5 Optimization

By Remark 5, solving (C) amounts to satisfying the two conditions of (5): a lasso in  $W^{(1)}$  and the stationarity of the Lagrangian (7) in  $\varphi$ . Each iteration takes one step on each (Section 5.1), and the iterations run along a path of increasing levels ending at  $\lambda_\alpha^{\text{DB}}$  (Section 5.2).

### 5.1 The Iteration

An iteration normalizes the network, then takes one step on each condition of (5). Let  $\mathcal{S}_\tau(x) = \text{sign}(x) \max(|x| - \tau, 0)$  denote soft-thresholding.

**1. Normalize.** The network is replaced by  $\mathcal{N}(\boldsymbol{\theta}) \in \Theta_1$  (Definition 2), which changes neither the fit nor the objective (Proposition 3). The gradients and the moments of Adam rescale with the weights and are rescaled along: divided by  $s_k$  (first moment) or  $s_k^2$  (second moment) for  $W_{k,\cdot}^{(1)}$  and  $b_k^{(1)}$ , multiplied likewise for  $W_{\cdot,k}^{(2)}$ . Without this step, the effective step on neuron  $k$  would scale like  $s_k^2$ , and a neuron whose sensitivity shrinks would freeze before its weights reach zero.

**2. Lasso step on the first layer.** The penalty is handled by a proximal step in the metric of Adam (Kingma and Ba, 2017; Yun et al., 2021):

$$W_{kj}^{(1)} \leftarrow \mathcal{S}_{D_{kj}\lambda}(W_{kj}^{(1)} - D_{kj} g_{kj}), \quad D_{kj} = \frac{\delta}{\sqrt{\hat{v}_{kj}} + \varepsilon}, \quad (10)$$

where  $\hat{v}_{kj}$  is the moving average of  $g_{kj}^2$ . A zero weight stays at zero if and only if  $|g_{kj}| \leq \lambda$ : the step size cancels, and the test is exactly (6). No momentum is used on  $W^{(1)}$ , so that the test bears on the current gradient.

**3. Lagrangian step on the other parameters.** The parameters  $\varphi$  take an Adam step along

$$\mathbf{d}_\varphi = \nabla_\varphi \ell + \sum_{k=1}^{p_1} \mu_k \nabla_\varphi s_k, \quad \mu_k = \lambda \|W_{k,\cdot}^{(1)}\|_1, \quad (11)$$

the gradient in  $\varphi$  of the Lagrangian (7) at its closed-form multipliers  $\nu_k = \mu_k$ .

The fixed points of the iteration are exactly the points satisfying (5), and at the null model the iteration applies the calibrated test.

**Proposition 8 (Fixed points)** *(a) A network  $\theta \in \Theta_1$  where no hidden pre-activation vanishes, with zero first moment on  $\varphi$ , is left unchanged by an iteration if and only if it satisfies (5).*

*(b) Under the assumptions of Theorem 6, the null network  $\theta^0$ , with zero first moment on  $\varphi$ , is left unchanged if and only if  $\lambda \geq \lambda_0(X, \mathbf{y})$ .*

By (b), the algorithm applies exactly the zero-thresholding test for which  $\lambda_\alpha^{\text{DB}}$  is calibrated. As usual for neural networks trained with Adam, no convergence result is claimed (Reddi et al., 2018).

## 5.2 A Reversed Warm Path

While the ultimate goal is to solve (C) at  $\lambda = \lambda_\alpha^{\text{DB}}$ , the calibrated level  $\lambda_\alpha^{\text{DB}}$  should not in general be imposed directly at initialization. The reason is that  $\lambda_\alpha^{\text{DB}}$  is calibrated from the first-order behavior of the network at the null model. A relevant variable may nevertheless have little or no marginal association with the response when its effect is purely nonlinear. Consequently, its relevance may not be visible from the first-order score at the null model. Starting the optimization directly at  $\lambda_\alpha^{\text{DB}}$  may then drive the first-layer weights to zero before the network has learned the nonlinear representation through which that variable becomes informative.

We therefore take the conservative approach of solving for a sequence of  $\lambda$ 's in the spirit of simulated annealing. Namely, we propose solving (C) along an increasing grid

$$0 < \lambda^{(1)} < \lambda^{(2)} < \dots < \lambda^{(M)} = \lambda_\alpha^{\text{DB}}.$$

Taking as initial parameter values the solution corresponding to the previous  $\lambda^{(i)}$  leads to a sequence of sparser approximating solutions until solving for  $\lambda_\alpha^{\text{DB}}$  at the last step. Along this path, the sensitivity profiles  $\mathbf{a}_k(\theta)$  evolve with the learned representation. Hence, once nonlinear structure has been learned, the adaptive first-layer score  $X_j^\top(\mathbf{a}_k(\theta) \odot \mathbf{r}(\theta))$  can reflect information that is absent from the marginal score at the null model. Increasing  $\lambda$  then progressively removes first-layer connections that are no longer supported by these learned, observation-specific scores.

Each value of  $\lambda$  defines a phase, which runs the iteration until the objective of (C) has stabilized. The whole procedure is summarized in Algorithm 1.

## 5.3 Implementation

Every quantity of an iteration comes from standard backpropagation.

**Sensitivities.** Each  $\eta_i$  depends on  $\mathbf{x}_i$  only, so  $\partial\ell/\partial\mathbf{h}_i^{(1)} = r_i \mathbf{a}_i$ : the sensitivities come with the backward pass of the loss, or, where  $r_i = 0$ , from (1) at the cost of one forward pass of the deeper layers.

---

**Algorithm 1** Fit along the warm path

---

**Require:**  $X$  (centred, unit second moment),  $\mathbf{y}$ , level  $\alpha$ , number of phases  $M$

- 1:  $\lambda_\alpha^{\text{DB}} \leftarrow (1 - \alpha)$ -quantile of  $\|X^\top \hat{\mathbf{r}}_0(\mathbf{Y}_0)\|_\infty$  over Monte Carlo draws  $\mathbf{Y}_0$  under  $H_0$
  - 2: **for**  $\lambda = \lambda^{(1)} < \dots < \lambda^{(M)} = \lambda_\alpha^{\text{DB}}$  (warm start) **do**
  - 3:   **repeat**
  - 4:      $\boldsymbol{\theta} \leftarrow \mathcal{N}(\boldsymbol{\theta})$ , rescaling the gradients and moments along
  - 5:     compute  $\mathbf{g}_k$  by (4) and  $\mathbf{d}_\varphi$  by (11)
  - 6:      $W^{(1)} \leftarrow$  lasso step (10);    $\varphi \leftarrow \text{Adam}(\varphi, \mathbf{d}_\varphi)$
  - 7:   **until**  $\ell + \lambda \|W^{(1)}\|_1$  has stabilized
  - 8: **end for**
  - 9: **return**  $\hat{S} = \{j : \hat{W}_{\cdot,j}^{(1)} \neq \mathbf{0}\}$
- 

**The extra gradient term.** With the activation patterns frozen, the network at observation  $i$  is the linear map  $\mathbf{u} \mapsto \mathbf{a}_i^\top \mathbf{u}$ . Feeding  $\mathbf{u} = \mu_k \text{sign}(a_{i_k^*}) \mathbf{e}_k$  at the observation  $i_k^*$  where  $s_k$  is attained gives the output  $\sum_k \mu_k s_k$ , whose gradient is the multiplier term of (11). It costs a forward and a backward pass of the deeper layers on  $p_1$  rows, so an iteration costs about one epoch of a standard network.

## 5.4 Optional Refitting on the Selected Model

After completion of the regularization path, the selected support is determined from the nonzero columns of  $\hat{W}^{(1)}$ . If desired, the network can subsequently be reduced to this selected set of input features, together with the removal of any hidden neurons that have become inactive. The dimensions of  $\hat{W}^{(2)}$  are adjusted accordingly. An optional final training phase can then be performed on this reduced architecture without regularization so as to reduce any estimation bias introduced by the penalized fit.

## 6 Experiments

### Appendix A. Proofs

**Proof of Proposition 4.** Near  $\boldsymbol{\theta}$  the activation patterns are constant, so that  $s_k$  depends neither on  $W^{(1)}$  nor on the biases, and depends on  $W^{(2)}$  only through its column  $W_{\cdot,k}^{(2)}$ , in which it is positively homogeneous of degree one:  $\langle W_{\cdot,k}^{(2)}, \nabla_{W_{\cdot,k}^{(2)}} s_k \rangle = s_k = 1$ . The constraint gradients  $\nabla_{\varphi} s_k$  are therefore linearly independent, and the Lagrange multiplier rule, for the smooth function  $\ell$  plus the convex penalty  $\lambda \|W^{(1)}\|_1$  under smooth equality constraints that do not involve  $W^{(1)}$ , gives multipliers  $\nu_k$  with the first condition of (5) and  $\nabla_{\varphi} \ell + \sum_k \nu_k \nabla_{\varphi} s_k = \mathbf{0}$ . It remains to show that  $\nu_k = \mu_k$ ; the same argument shows that  $\nu = \mathbf{0}$  forces  $W^{(1)} = \mathbf{0}$  when  $\lambda > 0$ .

Since  $\ell$  is unchanged by (3), differentiating in  $c$  at  $c = 1$  gives

$$\langle W_{k,\cdot}^{(1)}, \mathbf{g}_k \rangle + b_k^{(1)} \partial_{b_k^{(1)}} \ell - \langle W_{\cdot,k}^{(2)}, \nabla_{W_{\cdot,k}^{(2)}} \ell \rangle = 0.$$

Take the inner product of the stationarity condition with the vector equal to  $b_k^{(1)}$  in the coordinate of  $b_k^{(1)}$ , to  $-W_{\cdot,k}^{(2)}$  in those of  $W_{\cdot,k}^{(2)}$ , and zero elsewhere. By the identity above,

the  $\ell$  part gives  $-\langle W_{k,\cdot}^{(1)}, \mathbf{g}_k \rangle$ . Since  $s_j$  does not depend on the biases, and depends on  $W^{(2)}$  linearly through its column  $W_{\cdot,j}^{(2)}$  only, the multiplier part gives  $-\nu_k s_k = -\nu_k$ . Hence  $\langle W_{k,\cdot}^{(1)}, \mathbf{g}_k \rangle = -\nu_k$ . The first condition of (5) gives  $g_{kj} = -\lambda \text{sign } W_{kj}^{(1)}$  wherever  $W_{kj}^{(1)} \neq 0$ , so that  $\langle W_{k,\cdot}^{(1)}, \mathbf{g}_k \rangle = -\lambda \|W_{k,\cdot}^{(1)}\|_1$ , and  $\nu_k = \lambda \|W_{k,\cdot}^{(1)}\|_1 = \mu_k$ . If  $\nu = \mathbf{0}$ , then  $\|W_{k,\cdot}^{(1)}\|_1 = 0$  for every  $k$  when  $\lambda > 0$ , that is,  $W^{(1)} = \mathbf{0}$ .

**Proof of Theorem 6.** Write  $F(\boldsymbol{\theta}) = \ell(\boldsymbol{\theta}) + \lambda \|W^{(1)}\|_1$  and  $\boldsymbol{\theta} = (W^{(1)}, \varphi)$ , with  $\boldsymbol{\theta}^0 = (\mathbf{0}, \hat{\tau})$ . Since no hidden pre-activation vanishes at  $\boldsymbol{\theta}^0$ , the activation patterns are constant on a neighbourhood  $U$  of  $\boldsymbol{\theta}^0$ . On  $U$ ,  $f_{\boldsymbol{\theta}}(X)$  is a polynomial in  $\boldsymbol{\theta}$ ,  $\ell$  is twice continuously differentiable, and  $s_k$  does not depend on  $W^{(1)}$ .

(ii) Let  $\lambda < \lambda_0$  and pick  $(k, j)$  with  $|g_{0,kj}| = \lambda_0$ , by (8). Let  $\boldsymbol{\theta}(t)$  equal  $\boldsymbol{\theta}^0$  except for  $W_{kj}^{(1)} = -t \text{sign } g_{0,kj}$ ,  $t > 0$ . For small  $t$ ,  $\boldsymbol{\theta}(t) \in U$ , so that  $s(\boldsymbol{\theta}(t)) = s(\boldsymbol{\theta}^0) = \mathbf{1}$  and  $\boldsymbol{\theta}(t) \in \Theta_1$ . Then  $F(\boldsymbol{\theta}(t)) - F(\boldsymbol{\theta}^0) = -t\lambda_0 + \lambda t + O(t^2) < 0$  for small  $t$ :  $\boldsymbol{\theta}^0$  is not a local minimizer.

(i) Let  $\lambda > \lambda_0$  and  $\boldsymbol{\theta} = (W^{(1)}, \varphi) \in \Theta_1 \cap U$ . The point  $(\mathbf{0}, \varphi)$  lies in  $U$  as well if  $U$  is taken convex in  $W^{(1)}$ . A Taylor expansion in  $W^{(1)}$  gives, with a constant  $C$  uniform on a compact neighbourhood of  $\boldsymbol{\theta}^0$ ,

$$\ell(W^{(1)}, \varphi) \geq \ell(\mathbf{0}, \varphi) + \sum_k \langle \mathbf{g}_k(\mathbf{0}, \varphi), W_{k,\cdot}^{(1)} \rangle - C \|W^{(1)}\|_1^2.$$

At  $(\mathbf{0}, \varphi)$ , every observation has the same first-layer pre-activation, so that  $\mathbf{g}_k(\mathbf{0}, \varphi) = a_k(\varphi) X^\top \mathbf{r}(\varphi)$ , where  $\mathbf{r}(\varphi)$  is the gradient of  $\mathcal{L}_n$  at the constant output of  $(\mathbf{0}, \varphi)$ , and  $|a_k(\varphi)| = s_k(\mathbf{0}, \varphi) = s_k(\boldsymbol{\theta}) = 1$ , since  $s_k$  does not depend on  $W^{(1)}$  on  $U$ . As  $\varphi \rightarrow \hat{\tau}$ ,  $\|X^\top \mathbf{r}(\varphi)\|_\infty \rightarrow \lambda_0$ ; for  $\epsilon = (\lambda - \lambda_0)/2$  and  $\boldsymbol{\theta}$  close enough to  $\boldsymbol{\theta}^0$ ,  $\langle \mathbf{g}_k(\mathbf{0}, \varphi), W_{k,\cdot}^{(1)} \rangle \geq -(\lambda_0 + \epsilon) \|W_{k,\cdot}^{(1)}\|_1$ . Finally,  $\ell(\mathbf{0}, \varphi) \geq \ell(\boldsymbol{\theta}^0)$ , since the output of  $(\mathbf{0}, \varphi)$  is constant and  $\hat{\tau}$  gives the optimal constant fit. Altogether,

$$F(\boldsymbol{\theta}) - F(\boldsymbol{\theta}^0) \geq (\lambda - \lambda_0 - \epsilon) \|W^{(1)}\|_1 - C \|W^{(1)}\|_1^2 \geq 0$$

as soon as  $\|W^{(1)}\|_1 \leq \epsilon/C$ :  $\boldsymbol{\theta}^0$  is a local minimizer of (C).

**Proof of Proposition 8.** (a) On  $\Theta_1$ ,  $\mathcal{N}$  is the identity (Proposition 3(ii)). For scalars and  $D > 0$ ,  $x = \mathcal{S}_{D\lambda}(x - Dg)$  holds if and only if  $g = -\lambda \text{sign } x$  when  $x \neq 0$  and  $|g| \leq \lambda$  when  $x = 0$ , which gives the first condition of (5). On  $\varphi$ , the first moment becomes  $m' = \beta_1 m + (1 - \beta_1) \mathbf{d}_\varphi$  and the step is proportional to  $m'$ ; with  $m = \mathbf{0}$ , nothing moves if and only if  $\mathbf{d}_\varphi = \mathbf{0}$ , the second condition of (5).

(b) The network  $\boldsymbol{\theta}^0$  lies in  $\Theta_1$  and no hidden pre-activation vanishes there. By (a), it is left unchanged if and only if it satisfies (5), that is, by (9), if and only if  $\lambda \geq \lambda_0(X, \mathbf{y})$ .

## References

- Rina Foygel Barber and Emmanuel J Candès. Controlling the false discovery rate via knockoffs. *The Annals of Statistics*, 43(5):2055–2085, 2015.
- Alexandre Belloni, Victor Chernozhukov, and Lie Wang. Square-root lasso: pivotal recovery of sparse signals via conic programming. *Biometrika*, 98(4):791–806, 2011.
- Peter J. Bickel, Ya’acov Ritov, and Alexandre B. Tsybakov. Simultaneous analysis of lasso and dantzig selector. *The Annals of Statistics*, 37(4):1705–1732, 2009. doi: 10.1214/08-AOS620.
- Emmanuel Candès, Yingying Fan, Lucas Janson, and Jinchi Lv. Panning for gold: Model-X knockoffs for high-dimensional controlled variable selection. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 80(3):551–577, 2018. doi: 10.1111/rssb.12265.
- D. L. Donoho and I. M. Johnstone. Ideal spatial adaptation by wavelet shrinkage. *Biometrika*, 81(3):425–455, 1994.
- Jean Feng and Noah Simon. Sparse-input neural networks for high-dimensional nonparametric regression and classification. 2019.
- Caroline Giacobino, Sylvain Sardy, Jairo Diaz-Rodriguez, and Nick Hengartner. Quantile universal threshold. *Electronic Journal of Statistics*, 11(2):4701–4722, 2017. doi: 10.1214/17-EJS1362.
- Diederik Kingma and Jimmy Ba. Adam: A method for stochastic optimization. 2017.
- Ismael Lemhadri, Feng Ruan, Louis Abraham, and Robert Tibshirani. Lassonet: A neural network with feature sparsity. *Journal of Machine Learning Research*, 22(127):1–29, 2021.
- Christos Louizos, Max Welling, and Diederik P. Kingma. Learning sparse neural networks through  $l_0$  regularization. 2018. URL <https://arxiv.org/abs/1712.01312>.
- Yang Lu, Yingying Fan, Jinchi Lv, and William Stafford Noble. Deeppink: reproducible feature selection in deep neural networks. In *Advances in Neural Information Processing Systems*, volume 31, pages 8676–8686, 2018.
- Behnam Neyshabur, Ruslan Salakhutdinov, and Nathan Srebro. Path-sgd: Path-normalized optimization in deep neural networks. In *Advances in Neural Information Processing Systems (NIPS)*, volume 28, pages 2422–2430, 2015a. URL <https://papers.nips.cc/paper/5797-path-sgd-path-normalized-optimization-in-deep-neural-networks>.
- Behnam Neyshabur, Ryota Tomioka, and Nathan Srebro. Norm-based capacity control in neural networks. In Peter Grünwald, Elad Hazan, and Satyen Kale, editors, *Proceedings of the 28th Conference on Learning Theory (COLT)*, volume 40 of *Proceedings of Machine Learning Research*, pages 1376–1401. PMLR, 2015b. URL <https://proceedings.mlr.press/v40/Neyshabur15.html>.

- Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of Adam and beyond. In *International Conference on Learning Representations (ICLR)*, 2018.
- Sylvain Sardy, Maxime van Cutsem, and Sara van de Geer. The pivotal information criterion. *ArXiv*, arXiv:2603.04172, 2026.
- Pierre Stock, Benjamin Graham, Rémi Gribonval, and Hervé Jégou. Equi-normalization of neural networks. In *International Conference on Learning Representations (ICLR)*, 2019.
- Robert Tibshirani. Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society. Series B (Methodological)*, 58(1):267–288, 1996.
- Maxime van Cutsem and Sylvain Sardy. Survival of the fittest cox model: Pivotal variable selection for time-to-event data. *ArXiv*, arXiv:2510.19374, 2026.
- Yutaro Yamada, Ofir Lindenbaum, Sahand Negahban, and Yuval Kluger. Feature selection using stochastic gates. In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 10648–10659. PMLR, 2020.
- Liu Yang, Jifan Zhang, Joseph Shenouda, Dimitris Papailiopoulos, Kangwook Lee, and Robert D. Nowak. PathProx: A proximal gradient algorithm for weight decay regularized deep neural networks, 2022. Short version: A Better Way to Decay: Proximal Gradient Training Algorithms for Neural Nets, NeurIPS 2022 Workshop OPT.
- Jihun Yun, Aurelie C Lozano, and Eunho Yang. Adaptive proximal gradient methods for structured neural networks. *Advances in Neural Information Processing Systems*, 34: 24365–24378, 2021.
- Bo Zhao, Nima Dehmamy, Robin Walters, and Rose Yu. Symmetry teleportation for accelerated optimization. In *Advances in Neural Information Processing Systems*, volume 35, 2022.